Aplicación para la gestión de pedidos en una empresa de publicidad

BEIM Track

Jorge Junior Miranda Gómez

Haider Stive Pava López

Harold David Hernández Vásquez

Fundación Universitaria para el Desarrollo Humano

UNINPAHU

Facultad de Ingeniería y Tecnologías de la Información

Ingeniería de Software

Bogotá D.C.

2026

Tabla de contenido

Arquitectura Propuesta 3
Visión general 3
Elementos de la arquitectura 5
Comunicación entre components 6
Ejemplo creación de un pedido 6
Componentes del backend y sus responsabilidades 7
Despliegue 8
Justificación de la arquitectura 8
Trazabilidad con los requerimientos 9

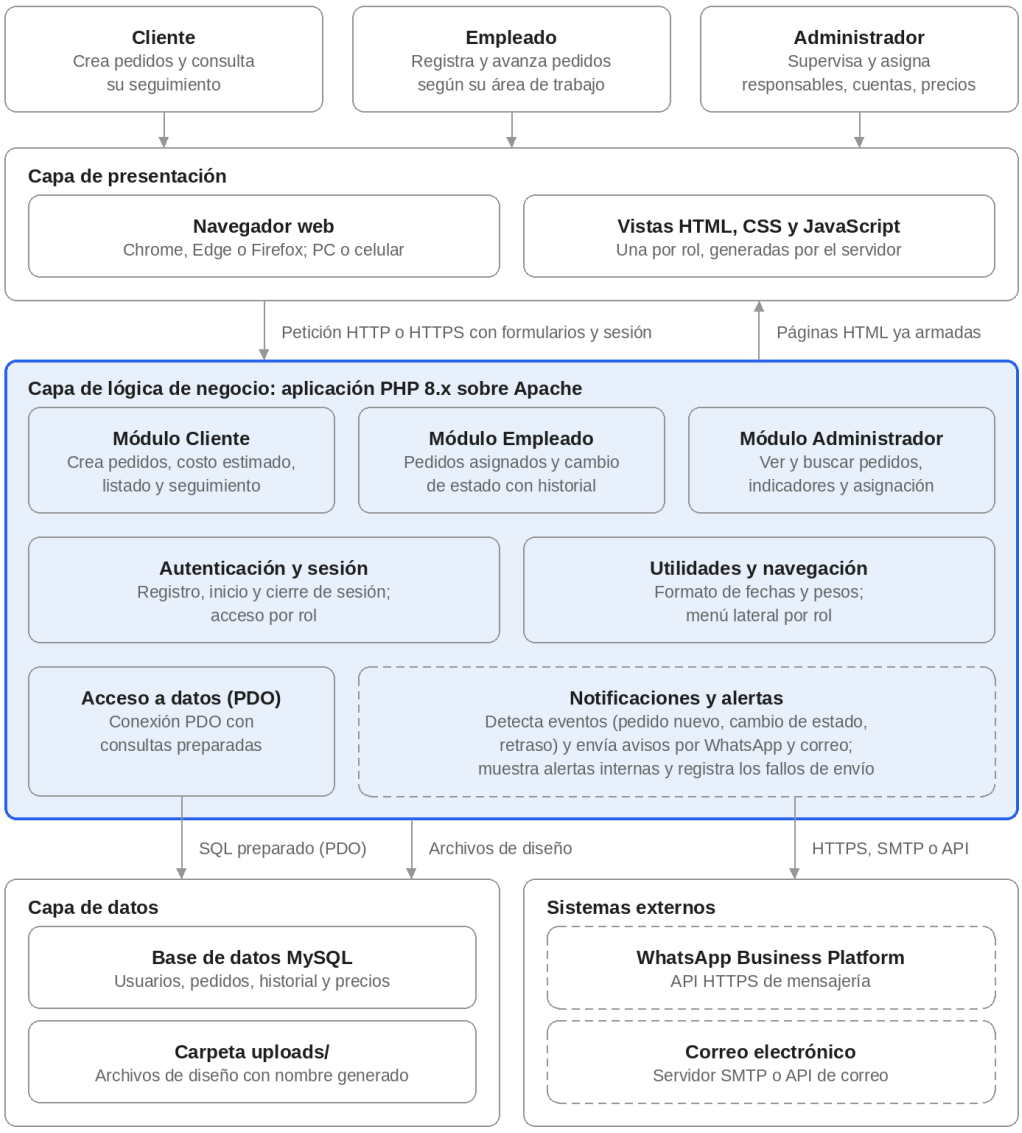
Arquitectura Propuesta

Visión general

BEIM Track se construye como una **aplicación web monolítica cliente-servidor en tres capas**:

- **Capa de presentación:** el navegador del usuario y las páginas HTML, CSS y JavaScript que el servidor genera para cada rol.
- **Capa de lógica de negocio:** la aplicación PHP que valida los datos, aplica las reglas del negocio, controla el acceso por rol y gestiona los pedidos.
- **Capa de datos:** la base de datos MySQL y la carpeta de archivos de diseño.

Todo el sistema se despliega como una sola aplicación web, junto con su base de datos, en un único servidor. Hacia afuera, la aplicación consume dos servicios externos de notificaciónes


Cliente
Crea pedidos y consulta su seguimiento
Empleado
Registra y avanza pedidos según su área de trabajo
Administrador
Supervisa y asigna responsables, cuentas, precios
Capa de presentación
Navegador web
Chrome, Edge o Firefox; PC o celular
Vistas HTML, CSS y JavaScript
Una por rol, generadas por el servidor
Petición HTTP o HTTPS con formularios y sesión
Páginas HTML ya armadas
Capa de lógica de negocio: aplicación PHP 8.x sobre Apache
Módulo Cliente
Crea pedidos, costo estimado, listado y seguimiento
Módulo Empleado
Pedidos asignados y cambio de estado con historial
Módulo Administrador
Ver y buscar pedidos, indicadores y asignación
Autenticación y sesión
Registro, inicio y cierre de sesión; acceso por rol
Utilidades y navegación
Formato de fechas y pesos; menú lateral por rol
Acceso a datos (PDO)
Conexión PDO con consultas preparadas
Notificaciones y alertas
Detecta eventos (pedido nuevo, cambio de estado, retraso) y envía avisos por WhatsApp y correo; muestra alertas internas y registra los fallos de envío
SQL preparado (PDO)
Archivos de diseño
HTTPS, SMTP o API
Capa de datos
Base de datos MySQL
Usuarios, pedidos, historial y precios
Carpeta uploads/
Archivos de diseño con nombre generado
Sistemas externos
WhatsApp Business Platform
API HTTPS de mensajería
Correo electrónico
Servidor SMTP o API de correo

Elementos de la arquitectura

Elemento	En BEIM Track	Tecnología
Usuarios o actores	**Cliente:** crea pedidos, consulta su seguimiento y recibe avisos. **Empleado** (áreas Recepción, Diseño, Producción y Calidad): registra y avanza pedidos y recibe alertas. **Administrador:** supervisa, asigna responsables y gestiona cuentas y precios.	Acceso por navegador; avisos por WhatsApp y correo
Aplicaciones	Navegador web del usuario y la aplicación web BEIM Track alojada en el servidor.	Chrome, Edge o Firefox; PHP 8.x sobre Apache
Frontend	**Vistas por rol:** inicio de sesión y registro, **panel del cliente**, formulario de pedido con costo estimado en vivo, seguimiento del pedido, **panel del empleado y panel del administrador,** con las alertas internas según el rol. Menú lateral común y una sola hoja de estilos adaptable a celular.	HTML5, CSS3, JavaScript sin librerías
Backend	Aplicación PHP organizada en módulos por rol. Ejecuta toda la lógica, detecta los eventos que generan notificaciones y se comunica con los servicios externos. El navegador nunca habla con ellos.	PHP 8.x
Servicios	Servicios internos de la aplicación: autenticación y sesión, gestión de pedidos, seguimiento e historial, asignación de responsables, gestión de archivos y **notificaciones y alertas**.	Módulos PHP
Base de datos	Almacena usuarios, pedidos, historial de cada pedido y precios por categoría. Para las notificaciones se agrega una tabla de alertas y de envíos, con su estado y sus fallos. Garantiza la integridad con llaves foráneas y valores restringidos.	MySQL (InnoDB)
Almacenamiento de archivos	Carpeta uploads/ del servidor donde se guardan los archivos de diseño adjuntos a los pedidos, con nombre generado por el sistema.	Sistema de archivos del servidor
APIs	**Consume** dos APIs externas por HTTPS: WhatsApp Business Platform y un servicio de correo SMTP.	HTTPS, SMTP
Sistemas externos	**WhatsApp Business Platform:** mensajes a clientes y personal.	WhatsApp Business Platform;

	Servicio de correo electrónico: los mismos avisos por correo. Requieren cuenta empresarial, credenciales y conexión a internet.	SMTP o API de correo

Comunicación entre components

Origen → Destino	Medio	Qué viaja
Actor → Navegador	Interfaz gráfica	Clics, formularios y archivos a adjuntar
Navegador → Aplicación web	HTTP o HTTPS, GET y POST	Credenciales, datos del pedido, cambios de estado y la cookie de sesión
Aplicación web → Navegador	HTTP o HTTPS	Páginas HTML ya armadas, hoja de estilos y JavaScript, con las alertas internas
Aplicación web → Base de datos	PDO con consultas preparadas	Consultas SQL parametrizadas y sus resultados, incluidas las alertas y el registro de envíos
Aplicación web → Archivos	Lectura y escritura en el sistema de archivos	Archivos de diseño de los pedidos
Aplicación web → WhatsApp Business Platform	HTTPS	Mensaje de aviso al cliente o al personal: pedido registrado, cambio de estado, retraso
Aplicación web → Servicio de correo	SMTP de correo por HTTPS	Correo de aviso con los mismos eventos

Ejemplo creación de un pedido

- El cliente llena el formulario; el JavaScript calcula el costo estimado en pantalla.
- El navegador envía el formulario por POST con la cookie de sesión.
- La aplicación comprueba que haya sesión activa y que el rol sea cliente.
- La aplicación valida los datos y guarda el archivo adjunto en uploads/.
- La aplicación inserta el pedido en estado «Recibido» y su primera entrada de historial en la base de datos.
- La aplicación redirige al panel del cliente, que muestra el pedido nuevo.
- El módulo de notificaciones detecta el evento «pedido registrado» y envía el aviso por WhatsApp y correo. Si el envío falla, el fallo se registra para su revisión y el pedido ya queda guardado

Componentes del backend y sus responsabilidades

Componente	Archivos	Responsabilidad	Requerimientos
Autenticación y sesión	index.php, login_process.php, register_process.php, logout.php, includes/auth.php	Registrar clientes, iniciar y cerrar sesión, guardar el rol en la sesión y bloquear el acceso a páginas de otro rol	RF-01 a RF-04, RNF-01
Módulo Cliente	cliente/panel.php, crear_pedido.php, guardar_pedido.php, estado_pedido.php	Crear pedidos, calcular el costo estimado, listar los pedidos propios y mostrar su seguimiento e historial	RF-06, RF-08, RF-13, RF-14
Módulo Empleado	empleado/panel.php, actualizar_estado.php	Mostrar los pedidos asignados y cambiar su estado, registrando el cambio en el historial	RF-18, RF-20
Módulo Administrador	admin/panel.php, asignar_empleado.php	Ver y buscar todos los pedidos, mostrar indicadores y asignar responsables	RF-17, RF-22, RF-24
Notificaciones y alertas	Por crear	Detectar los eventos (pedido registrado, cambio de estado, retraso, atención de un usuario), enviar avisos por WhatsApp y correo, mostrar alertas internas según el rol y registrar los envíos fallidos	RF-26 a RF-29
Utilidades	includes/helpers.php	Funciones comunes de formato de fechas y valores en pesos	RNF-13
Navegación	includes/nav_sidebar.php	Menú lateral según el rol del usuario	RF-04
Acceso a datos	config/db.php	Abrir la conexión PDO a la base de datos; todas las consultas son preparadas	RNF-02
Configuración (parcial)	config/ y .env (por crear)	Leer de variables de entorno las credenciales de la base de datos y de los servicios externos; el .env no se sube al repositorio	RNF-11, RNF-15

Presentación	css/style.css	Estilos compartidos y adaptación a pantallas pequeñas	RNF-08
Archivos	uploads/	Guardar los archivos de diseño de los pedidos	RF-11

Despliegue

La aplicación, la base de datos y la carpeta de archivos se instalan en un solo servidor

- **Desarrollo y revisión:** equipo local con XAMPP, con phpMyAdmin para administrar la base de datos. La base de datos se crea importando database/schema.sql.
- **Prueba piloto con la empresa:** un computador de la empresa con PHP y MySQL
- **Servicios externos:** requieren conexión a internet, una cuenta empresarial de WhatsApp y las credenciales del correo.
- **Configuración:** las credenciales de la base de datos y de los servicios externos se leen de variables de entorno (archivo .env, no versionado).

Justificación de la arquitectura

La arquitectura responde al problema de la empresa, que es la falta de un registro único y de seguimiento de los pedidos, por estas razones:

- **Un solo punto de verdad.** Al centralizar pedidos, estados e historial en una base de datos compartida, desaparecen el papel y la comunicación de voz a voz. Cliente, empleados y administrador ven la misma información actualizada.
- **Datos relacionados.** Cliente, pedido, responsable e historial dependen unos de otros. Una base de datos relacional con llaves foráneas evita pedidos huérfanos y estados inválidos.
- **Reglas y seguridad en el servidor.** Las validaciones, el control de acceso por rol, los permisos y las credenciales de los servicios externos viven en el backend; el navegador solo muestra lo que el servidor decide. Esto sostiene los requisitos de seguridad y la regla de que cada usuario solo ve lo suyo.
- **Simplicidad acorde con el alcance.** Es una sola empresa, con tres tipos de usuario y un equipo de tres estudiantes. Un monolito en capas es suficiente, fácil de explicar, de instalar en cualquier computador y de mantener por módulos de rol.
- **Comunicación donde ya están las personas.** El problema de la empresa incluye la mala comunicación y los retrasos sin aviso. Un módulo de notificaciones, apoyado en WhatsApp y correo, le avisa al cliente y al personal sin que entren a la aplicación, y las alertas internas cubren lo que se atiende dentro de ella.
- **Dependencia externa controlada.** Los servicios de terceros se llaman solo desde el módulo de notificaciones. Si fallan, el pedido y su seguimiento siguen funcionando y el fallo queda registrado. El núcleo usa software libre; solo los mensajes pueden generar costos según el proveedor.

- **Evolución por iteraciones.** Como el código está separado por rol, las funciones de (estados por área, canal de recepción, retrasos y notificaciones) se agregan dentro de los módulos existentes, en línea con el enfoque iterativo del proyecto.

Trazabilidad con los requerimientos

☑ implementado ·

◍ parcial

☐ planeado

Requerimientos	Componente	Estado
RF-01 a RF-04	Autenticación y sesión	☑
RF-05	Módulo Administrador (gestión de cuentas)	☐
RF-06, RF-08	Módulo Cliente	☑
RF-07, RF-11	Módulo Cliente y archivos	◍
RF-09, RF-10	Módulos Cliente, Empleado y Administrador	☐
RF-12	Módulo Cliente	☐
RF-13, RF-14	Módulo Cliente	☑
RF-15	Todos los módulos	◍
RF-16	Módulos Administrador y Empleado	◍
RF-17	Módulo Administrador	◍
RF-18	Módulo Empleado	☑
RF-19, RF-21	Módulo Empleado	☐
RF-20	Módulo Empleado	◍
RF-22, RF-24	Módulo Administrador	☑
RF-23, RF-25	Módulo Administrador	☐
RF-26, RF-27 (nuevos): avisos por WhatsApp y por correo	Notificaciones y alertas	☐
RF-28 (nuevo): alertas internas	Notificaciones y alertas; paneles de cada rol	☐

RF-29 (nuevo): registro de envíos fallidos	Notificaciones y alertas	☐
RNF-15 (nuevo): credenciales por variables de entorno	Configuración	◍